

USENIX

THE ADVANCED COMPUTING
SYSTEMS ASSOCIATION

Trust Nothing: RTOS Security without Run-Time Software TCB

Eric Ackermann and Sven Bugiel, *CISPA Helmholtz Center for Information Security*

https://www.usenix.org/conference/usenixsecurity26/presentation/ackermann

This paper is included in the Proceedings of the
35th USENIX Security Symposium.

August 12–14, 2026 • Baltimore, MD, USA

ISBN 978-1-939133-58-8

Open access to the Proceedings of the
35th USENIX Security Symposium
is sponsored by



جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

ARTIFACT EVALUATED
USENIX
AVAILABLE

Trust Nothing: RTOS Security without Run-Time Software TCB

Eric Ackermann, Sven Bugiel
CISPA Helmholtz Center for Information Security

Abstract

Embedded devices face an ever-expanding threat landscape: vulnerabilities in application software, operating system kernels, and peripherals threaten the embedded device integrity. Existing computer-architectural defenses fully consider at most two of these threat vectors in their security model.

This paper aims at addressing this gap using a novel capability architecture. To this end, we combine a *token* capability approach suitable for building an untrusted operating system with protection against malicious devices without requiring hardware changes to peripherals.

First, we develop and evaluate a full FPGA implementation of our capability architecture around legacy hardware components. Further, we present a soft real-time operating system based on Zephyr that has *no run-time software TCB*. To this end, we *disaggregate* Zephyr's subsystems into small, mutually isolated components. All subsystems that exist at run time, including scheduler, allocator and DMA drivers, and all peripherals are *fully untrusted*. We believe that our work offers a foundation for more rigorous security-by-design in tomorrow's security-critical embedded devices.

1 Introduction

Owing to advances in computer architectures, inflexible and expensive hardware circuitry in embedded devices is increasingly being replaced with flexible software running on generic inexpensive systems-on-chip (SoCs). While reducing engineering cost and time to market, this *softwarization* has lead to an increase of *security vulnerabilities* in embedded devices that compromise integrity and availability of critical services. Vulnerabilities are commonly found in *application software*, the *operating system kernel*, or *peripheral devices with Direct Memory Access (DMA) facilities*.

Application software vulnerabilities in embedded devices are widespread: An exemplary study of printers found known vulnerabilities in 80% of the analyzed firmware images [26]. On a broader scale, existing attack taxonomies list thousands of known CVEs for embedded devices [57, 69]. Exploiting vulnerabilities in application software can be used as a stepping stone to compromise the entire device. To this end, adversaries can exploit vulnerabilities in the operating system kernel: In RTOS such as Zephyr, separation of kernel and user space is opt-in and not supported on all compatible SoCs [67]. Hence, *any software vulnerability causes the attacker to gain complete control over the device*. Even in Linux, where kernel and application software are separated using a memory management unit (MMU), vulnerabilities that allow kernel exploitation continue to be found [20, 21, 48, 66, 78]. Finally, *devices within the SoC* are also a possible threat vector: Beniamini [15] demonstrates an over-the-air exploitation of a broadcom mobile WiFi chip, turning a benign DMA device into a hardware trojan. Markettos et al. [50] showcase an implant that poses as a benign Thunderbolt-enabled peripheral, but performs DMA attacks via PCIe encapsulation.

All existing defenses against the three threat vectors—application software, kernel, devices—can only provide security against two out of three threat vectors. Generally, kernels utilize MMUs to provide security against compromised application software and devices. However, MMU designs require trusting the kernel to configure the MMU correctly, making it impossible to achieve *no run-time software TCB*. The goal of designing operating systems without trusted kernels is far from a new idea—in fact, Dennis and Van Horn [27] already addressed this very problem with their concept of *capability architectures* in 1966. Several modern capability architectures capitalize on this idea to reduce or even eliminate the trusted part of the operating system [33, 42, 76, 80]. However, they share the limitation that they *exclude devices from the threat model*. Thereby, they trade a minimal *software TCB* for a larger *hardware TCB* or need to *trust their drivers* [22]. As a result, no existing architecture achieves security in the presence of both untrusted OSes and devices simultaneously.

Recently, Ackermann et al. [2] proposed the Northcape concept for a novel capability architecture that closes this gap. Northcape considers all application software, the kernel and devices possible threats. The key idea is to implement capabil-

ity memory access control at the *system bus level*. However, before our work, Northcape existed purely as a theoretical concept without implementation and proof of feasibility.

Contribution Our work aims at answering the following research question: *How can an embedded system provide security against malicious application software, operating system kernels and peripheral devices without relying on a run-time software TCB while retaining soft real-time guarantees?* We solve this question using a hardware-software-co-design that comprises both the *Bredi SoC* and the *Skadi RTOS*.

First, we present Bredi (Breði) in Section 4. Bredi is the first hardware implementation of Northcape, built around the cva6 RISC-V CPU [82] and an unmodified ethernet subsystem [5] with DMA [3], targeting an FPGA. Bredi addresses two security shortcomings of Northcape: First, akin to CHERI-TrEE [70], it relied on trusted software for interrupt handling. Through modifications of cva6, we eliminate this trusted software, facilitating untrusted ISRs. Real-time availability was also out of scope for Northcape. Using non-maskable interrupts, we ensure the availability of critical services in the presence of attackers. Bredi also increases the security of Northcape's *exclusive access* and addresses *capability tag guessing attacks* using our novel *subsystem restrictions*.

Second, we present Skadi (Skaði) OS in Section 5. Skadi is a modified Zephyr RTOS [83] that uses the operations provided by Northcape to enforce *run-time isolation* of operating system components into *subsystems* that are *mutually distrusting*. Using Bredi operations, Skadi facilitates *full spatial and temporal isolation* for all data structures that are private to the subsystems or shared between the subsystems, including *hardware-enforced exclusive access* to any data structure. A *subsystem call* primitive allows synchronous function invocation between subsystems by *atomically* changing the set of accessible capabilities and control flow. Skadi implements all operating system services as *isolated subsystems*, leaving *no kernel* and *no run-time software TCB*. Peripheral devices, drivers and ISRs are also removed from the TCB. Skadi and Bredi are available under an OpenSource license [1].

Finally, we discuss the security and performance of our prototype in Section 6 and Section 7. The results show that in an ideal scenario such as a compute benchmark, Skadi is as fast as an insecure reference. For I/O-bound scenarios, while Skadi and Bredi currently impose an overhead on response times, the performance of the system remains practical for real-world use. As *security* was the focus of this work, Skadi and Bredi leave room for future performance optimization at the hardware and software levels. Overall, we believe that Skadi and Bredi can contribute to a persistent improvement of the security posture of tomorrow's embedded devices such as network infrastructure devices by realizing security-by-design for both the hardware and software stack.

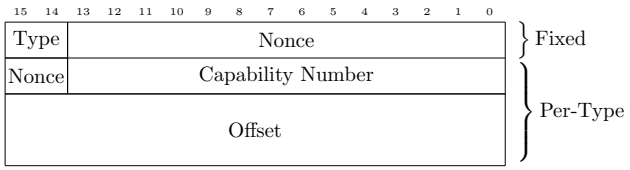


Figure 1: Northcape token format (32-bit offset type shown).

2 Background: The Northcape Architecture

We start with a brief introduction of object capability systems in Section 2.1. We then introduce Northcape, covering capability representation and translation in Section 2.2 and operations in Section 2.3. In this section, we present Northcape as implemented in Bredi. We discuss improvements made to Northcape [2] in Section 8.1.

2.1 Object Capability Systems

First coined by Dennis and Van Horn [27] in 1966, object capability architectures separate the physical memory of a computer into *segments* with byte-granular starting and ending addresses. Subjects (subsystems) need to hold explicit access rights (*capabilities*) for each object (segment) that they access. *Unforgeability* of capabilities ensures security.

Capability architectures do not require privileged modes of operation. Instead, an explicit *subsystem call* instruction transfers control between subsystems with mutual isolation.

Finally, *capability delegation* from one subsystem to another allows controlled sharing of *specific* data structures, eliminating *ambient authority* [55]. Thereby, delegation and subsystem calls facilitate cooperation between mutually isolated subsystems, enabling *untrusted kernels* [33].

2.2 The Northcape Capability Architecture

Representations. Two distinct representations of object capabilities have emerged in the literature: 1) *Fat-pointer systems* like CHERI [76] differentiate between capabilities and memory addresses. Capabilities need to be loaded into explicit *capability registers* and referenced as source or destination addresses in CPU instructions. CHERI capabilities *themselves* contain segment bounds and access permissions. Unforgeability of capabilities relies on *tagging* of memory locations that hold valid capabilities, requiring a separate *tag bitmap*. 2) *Token capability systems* like RV-CURE [42] separate capability metadata and representation. RV-CURE embeds a capability identifier into the high order bits of a pointer. The identifier is resolved to segment bounds via a central metadata table (CMT). Unforgeability of capabilities relies on a *MAC tag*.

Similar to RV-CURE, Northcape is a token-based capability system. As illustrated in Figure 1, Northcape separates 64-bit pointers into an *offset* into a capability in the low-order

bits and a *nonce* and *capability number* in the high bits. The number identifies a CMT entry, which holds the metadata.

Northcape makes the sizes of offset and number *variable* by introducing *offset types*, facilitating a tradeoff between the maximum size and number of supported capabilities. We support offsets from 8 to 32 bits in 8-bit increments, resulting in 2^{14} capabilities with 32 bit offset down to 2^{38} capabilities with 8 bit offset *at the same time*. We accounted for changing these values with minimal effort in our prototype.

The separation of Northcape capability tokens into offset, number and nonce makes it possible to use them interchangeably with C pointers. Crucially, one can *interpret* a capability token as the *starting position* of an object in memory. The fields of the objects can then be addressed by incrementing the offset. Therein, this adjustment of the offset happens *transparently*, facilitating *backwards compatibility* with legacy pointer arithmetic. In contrast, RV-CURE and CHERI require the use of explicit *instructions* for accessing capabilities [42, 76].

Capability Types. In the following, we give a textual description of Northcape's capability types and their relationships as illustrated in Figure 7 in the appendix.

Northcape can ensure *sub-object protection*, a common feature in modern object capability systems [42, 76, 80]. Therefore, it distinguishes *direct capabilities* and *indirect capabilities*: Direct capabilities have ownership of the segment that they resolve to. Northcape operations ensure that all existing direct capabilities are *disjoint*.

Indirect capabilities grant access to a subsegment of a direct capability. Indirect capabilities for the same direct capability may overlap, and can be created *recursively*: One can create an indirect capability from an indirect capability, decreasing segment bounds (*monotonicity* [76]). Northcape tracks the parent of each indirect capability for *temporal security*.

Using the *lock* operation, subsystems can gain *exclusive access* to capabilities. To achieve this, *lock* creates a *lock-holder capability*. Like indirect capabilities, lock-holder capabilities can permit access to a subset of a parent direct or indirect capability. However, while the lock-holder exists, *access to all overlapping capabilities is impossible*.

Capability Unforgeability. The security of capability systems hinges on the *unforgeability* of capabilities: A security policy can only be enforced if subsystems are confined to the capabilities that were explicitly delegated to them [27, 55]. Northcape achieves unforgeability using two complementary layers of security: *subsystem restrictions* and a *nonce*.

The 16-bit random nonce is a part of each capability token. Hence, forgery of a capability requires guessing both the nonce and number. Our prototype generates nonces using a pseudo-random number generator that is initially seeded with entropy from a hardware true random number generator [53]. However, because capability numbers can be predictable, guessing a capability token has a likelihood of $\frac{1}{2^{16}}$. Related capability systems like RV-CURE rely *solely* on the low likelihood of guessing the nonce for security [42]. However, Northcape provides a second layer of unforgeability:

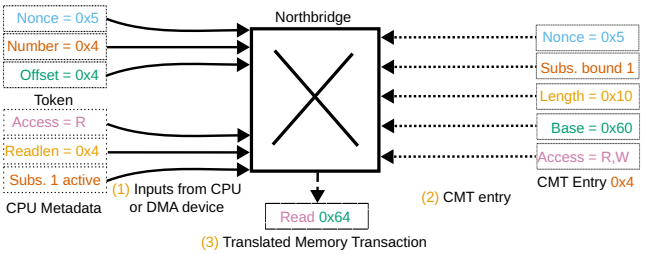


Figure 2: Translation of a Northcape capability token.

Capabilities can optionally carry a *subsystem-ID bound restriction* in their metadata. When a subsystem-ID bound capability is accessed, Northcape ensures that the 32-bit subsystem ID in the capability matches the active subsystem. To this end, Northcape tracks the ID of the active subsystem across subsystem calls. Furthermore, a 16-bit device identifier is compared with the identity of the current device. Non-CPU devices like DMA controllers always run an implicit task 0. Thereby, when a *different subsystem* guesses a nonce, it is unable to access the capability due to its mismatching subsystem ID. Importantly, the ID does not form an ambient authority since all access is still only possible through capabilities [55].

Even in the presence of subsystem restrictions, the nonce adds additional utility and *defense-in-depth*: The nonce is still useful for delegating a capability to multiple or unknown recipients or devices, who can add subsystem restrictions later using the *restrict* and *lock* operations. The nonce also *randomizes* the address of each object at no additional run-time cost. Thereby, Northcape enforces a strong form of *address space layout randomization* for all software and devices.

Capability Translation. Figure 2 illustrates the resolution of capabilities to physical addresses: Software on a CPU or a DMA controller starts a memory transaction via the system bus (1). The Northcape-enabled northbridge will *interpret* the provided address as a capability token and parse the number, offset and nonce components. The number field uniquely identifies a CMT entry. The northbridge will retrieve this entry (2). First, it will ensure *unforgeability* by verifying the nonce and any subsystem ID restrictions. It will then determine whether the requested *offset* and *length* remain within the bounds of the capability, ensuring *spatial security*. If the resolved capability is indirect or a lock-holder, the northbridge will *recurse* until reaching the *direct capability*, ensuring all parents are valid and unlocked. This is crucial for *temporal security*, specifically *revocation* and *exclusive access*. Finally, the northbridge will compute a physical address as the *base* stored in the CMT entry plus the token's *offset* (3).

Capability translation is a *speculation barrier*: without the CMT entry, the physical address is unknown. This prevents the hardware from allowing physical loads and stores before

ensuring validity of the capability. We discuss how the translation can be sped up using caching in Section 4.1.

2.3 Northcape Capability Operations

We will introduce Northcape's operations using three motivating examples: sharing of memory, dynamic memory allocation, and control transfer. The full list of operations with detailed descriptions is given in Table 11 in the appendix.

Secure Memory Sharing. Consider DMA transfers: A driver indicates a location *addr* and length *l* to a device. The device is assumed to transfer up to *l* bytes to or from *addr* before signalling completion. A different subsystem can perform a subsystem call into the DMA driver to initiate a transaction. The call can pass a capability token as an argument. The calling subsystem can *delegate* any subset of any capability that it has access to to the DMA driver. This is done using the *derive* or *clone* operations, which create *indirect capabilities* from a capability. Therein, *derive* allows shrinking capability bounds while *clone* preservers them. Additionally, bounds can be shrunk and access rights can be removed using *restrict*. The *inspect* operation allows the DMA driver to determine metadata of a capability for sanity checks.

Subsystem-ID protections comprise both the *subsystem ID* and the *device ID*. Thereby, DMA devices cannot access *any* capability with subsystem-ID protection, even with the correct capability token. This enforces DMA isolation.

The DMA driver can delegate (a subset of) the received capability to the DMA device. To this end, it creates a capability with a subsystem ID protection suitable for the device and writes the capability token into the existing address registers. As Northcape capability tokens are encoded to resemble unprotected bus addresses, no modification to the DMA device is necessary. Northcape will ensure *spatial security*, i.e., confine the DMA device to the bounds of the capability with byte granularity. After completion of the transfer, the driver can destroy the delegated capability using *drop*, which only succeeds for capabilities without living children and invalidates the capability, ensuring *temporal security*.

In some cases, such as the DMA descriptors used by Xilinx' DMA [3], the DMA and its driver take turns reading and writing the same object. Here, the DMA driver can use *lock* to gain exclusive access to the descriptor, preventing time of check to time of use (TOCTOU) attacks by the device.

Memory Management. As noted earlier, memory is owned exclusively by *direct capabilities*. This is useful for slab allocators. The allocator can be provided a large direct capability as an initial heap. When subsystems request memory via a subsystem call, the allocator can use *create* to slice off direct capabilities from the heap. Thereby, allocated capabilities can be locked without the allocator losing access to the heap.

Freed memory can be returned to the heap using a *merge* operation that combines two adjacent capabilities into one and restores access permissions. Together, *create* and *merge* are sufficient for implementing a slab allocator [81].

Finally, different from *create*, the *revoke* operation can be used on a direct capability *that is locked or has children*. It *overwrites* the associated segment with zeros, ensuring that no secrets are leaked, and creates a new output capability with no children. *revoke* is useful, e.g., for reclaiming memory of subsystems after a crash by an *untrusted allocator*.

Control Transfer with Mutual Isolation. Finally, Northcape provides a *subsystem call* operation *calls*, which allows *atomically* changing both the program counter of a CPU and the active subsystem ID, changing the sphere of protection. *calls* can only be invoked on the *first byte* of capabilities with special *set-subsystem-ID* restrictions: Consequently, *calls* allows transfers to explicit *entry points* while ensuring a subsystem ID cannot be *hijacked* by a different subsystem.

Set-subsystem-ID restrictions need to be assigned when a capability is created. Crucially, subsystems can only create set-subsystem-ID restrictions for their own subsystem ID. Otherwise, it would be trivial to hijack foreign subsystem IDs. We make one exception to this rule: a special subsystem ID 0 can create *arbitrary* set-subsystem-ID restrictions. Conceptually, subsystem ID 0 is exclusively used by the trusted Skadi Loader, which needs this privilege to bootstrap the remaining subsystems. The loader makes subsystem ID 0 inaccessible at the end of load time, preventing abuse.

3 Security Model

Lefeuvre et al. [45] propose the following security model for a compartmentalization mechanism: **C** (Confidentiality): A subject cannot *read* out of its protection domain, **I** (Integrity): A subject cannot *write* out of its protection domain, **A** (Availability): A subject cannot *prevent* other protection domains from executing normally. We adapt this security model to the combined system comprising Skadi and Bredi. Therein, our subjects refer to *subsystems*. The *protection domain* (in capability terminology: *sphere of protection* [27]) of a subsystem entails all capabilities that were *explicitly delegated* to it. This comprises per-subsystem code and data segments as well as any capabilities received as arguments to subsystem calls. For **C** and **I**, we guarantee *spatial* and *temporal* security without requiring explicit *memory sweeps* [77]. Finally, we extend the definition of **A** to include *soft real-time execution*.

We further require *exclusive access* to capabilities for which properties **C** and **I** are to be protected: Subsystems can assume that they have exclusive access to private code and data segments by default, excluding subsets that they delegated to other subsystems explicitly. Subsystems can gain exclusive access to a shared capability using the *lock* operation, after which the mechanism must guarantee **C** and **I**. Furthermore, subsystems can use the *drop* operation to *un-share* a capability, regaining **C** and **I** for that capability.

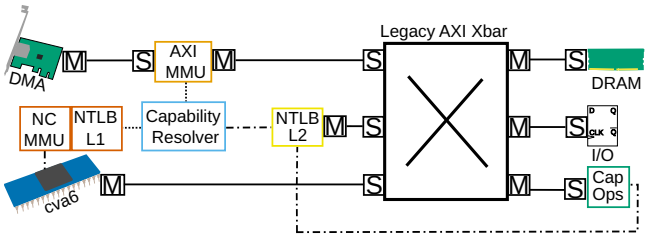


Figure 3: Bredi architecture. Solid lines denote AXI, dotted lines AXI-Stream, dash-dotted lines custom buses. NTLB L1 is further duplicated into instruction and data NTLB L1.

Let the hardware trusted computing base (TCB) refer to the CPU, capability-enforcing northbridge and the system memory including buses. The software TCB comprises solely the *Skadi loader*. The loader is destroyed at the end of the boot sequence, leaving *no run-time software TCB*. All *software and hardware* components outside of the TCB are assumed openly malicious. Side-channel and physical attacks are an orthogonal problem and thus excluded.

We further define the *availability TCB (aTCB)* as the TCB in conjunction with all software procedures involved in *servicing interrupts*. Note that the aTCB components that are not also in the TCB are only trusted for **A**—they are allowed to attack security goals **C** and **I**, e.g, by manipulating registers.

The **A** property is in conflict with our **I** property: A software subsystem which disables interrupts and proceeds into an infinite loop can prevent other subsystems from executing ever again. Hence, we need to be able to *preempt* subsystems and let other critical subsystems run. However, being able to *preempt* such an adversary requires being able to prevent it from disabling interrupts. In order to resolve this conflict without having to fall back to using CPU-enforced privilege rings, we relax our **A** goal: The property is only required if either the adversary *does not disable interrupts* or the availability-critical subsystem is an *interrupt handler*. We explicitly allow the availability-critical subsystem to *not be the first-level interrupt handler*, however, relying on the aTCB we defined above to guarantee availability on the interrupt handler chain.

4 Bredi SoC Implementation

This section covers our hardware implementation of Northcape, Bredi. We will briefly introduce Bredi's core components. We will then discuss three core features of Bredi that are crucial for security: First, Section 4.1 discusses how our caching hierarchy can hide the overhead of recursive capability resolution, which is critical for availability (**A**). Second, Section 4.2 discusses changes to interrupt handling that were necessary to exclude ISRs from the TCB, a necessity for confidentiality (**C**), integrity (**I**) and availability (**A**). Finally, Section 4.3 discusses custom CPU instructions needed for **A**.

Bredi is based on cva6 [82], a Linux-capable in-order application processor. We configure the RV64G instruction set (64-bit words, FPU) with a PMP and the sv39 MMU [63]. We incorporate the proprietary DMA-based Xilinx AXI Ethernet subsystem [3, 5] for evaluating DMA compatibility and performance (**A**). We also use Xilinx' AXI crossbar [4]. Furthermore, we have implemented components for Northcape support as detailed in Figure 3: a Northcape Memory Management Unit (NC MMU) with TLB (NTLB L1) for cva6, an AXI MMU, a Capability Resolver, Capability Operations (CapOps), and a level-2 Northcape TLB (NTLB L2).

AXI MMU. The AXI MMU enforces memory access restrictions on DMA devices, akin to an IOMMU or CHERI CapChecker [22]. It interfaces with a DMA device via its front-side bus (AXI 4 [11]), performs access control and translation from capability tokens to physical addresses as depicted in Figure 2 and forwards bus transactions to the AXI interconnect. Remember that the translation is *opaque* to the device.

The AXI MMU has no cache, relying on the capability resolver and NTLB L2 instead. This leads to a marginal performance degradation, but reduces chip area significantly.

cva6 MMU. Bredi's cva6 contains instruction and data Northcape MMUs comparable to the AXI MMU. The cva6 MMU contains a small, fully-associative TLB (NTLB L1) that maps capability tokens to bounds, permissions and restrictions and handles misses via the capability resolver.

Capability Resolver. All cva6 and AXI MMUs interface with a single Northcape Capability Resolver via a round-robin arbiter. In doing so, the capability resolution and parsing logic only needs to exist once, irrespective of how many MMUs exist in the system, reducing chip area. The resolver is pipelined, allowing it to accept one resolution request per cycle and send one resolution response per cycle. The resolver introduces two cycles of latency plus one cycle per NTLB L2 cache hit. As discussed in Section 4.1, our cache invariant allows us to skip all hierarchy levels if the first level encounters a hit.

Capability Operations Module. The penultimate component in the Northcape system is the operations module, which implements the Northcape operations that we introduced in Section 2.3 in a hardware finite state machine. To this end, it is the only device in the system with write access to the CMT via the NTLB L2. Devices and software can access it via MMIO. The MMIO interface is *locked* to a subsystem ID after a subsystem starts setting up an operation, ensuring that concurrent devices cannot disturb the interaction.

NTLB L2. Finally, the operations module and resolver share the set-associative write-through dual-port NTLB L2. In contrast to the NTLB L1, the NTLB L2 maps capability tokens to the *full CMT entry*. Thereby, both the capability resolver and operations unit can operate on NTLB L2 entries and need not access the CMT in memory, reducing memory accesses significantly. The dual-port access alleviates the need for cache arbitration, benefiting availability (**A**).

4.1 Contribution 1: Cache Hierarchy (A)

As detailed in Section 2.2, Northcape capabilities can be direct or indirect. CMT entries provide all necessary information for translating a capability token and enforcing segment bounds, permission and restriction enforcement. In effect, CMT entries can be *cached* like page table entries in a TLB.

Hence, the cva6 MMU has a small, fully-associative first-level NTLB. The NTLB L1 maps capability IDs to bounds, restrictions, and permissions and thereby facilitates completing the address translation process within one cycle. Thus, cache hits *do not block cva6' pipeline*, akin to the TLB used in the sv39 MMU [63]. Our evaluation in Section 7.5 shows that high NTLB L1 hit rates provide performance indistinguishable from a baseline scenario *without memory protection*.

However, NTLB L1 entries can become *stale* after completion of a Northcape operation such as *lock* or *drop*. Stale entries can be used to read or write data outside of the *sphere of protection* of a subsystem, jeopardizing **C** and **I**. Our implementation destroys stale NTLB L1 entries at the end of each operation to ensure **C** and **I**. To this end, the operations module broadcasts the IDs of all modified capabilities to the L1 NTLBs. If this capability is part of the cache, it is removed, prompting a new recursive resolution via the resolver. In case of *revoke* or *lock*, the entire NTLB L1 is invalidated, making sure no overlapping capabilities of the target are contained.

The NTLB L2 can also contain *stale entries*. However, the NTLB L2 cache is *write-through*. All operations that only affect the input capability and its direct parents (*derive*, *drop*, *clone*, *create*, *merge*, *inspect*, *restrict*) *cannot create stale entries*, which is why the cache does not need to be invalidated after these operations. On the other hand, *lock* and *revoke* can affect *any overlapping capability* of the input capability. Hence, these operations can leave *stale entries*, which need to be removed to ensure **C** and **I**.

A simple solution for the stale entries problem in the NTLB L2 is *flushing the cache* on *lock* and *revoke*. This provides full security at the expense of performance: capabilities would be evicted from the cache hierarchy as *collateral damage*. We provide the following solution that benefits **A**: After *lock* or *revoke*, we taint *all cache entries* as stale. After every NTLB L2 hit on a stale entry, we force the capability resolver to perform a hierarchical resolution of the capability. This is faster with stale entries than having to fetch all capabilities from memory. Bredi waits for the translation to complete before commencing the memory access, preventing speculative execution attacks. If the entry is valid, we remove the *stale* taint. Otherwise, we remove the stale entry from the cache.

4.2 Contribution 2: Untrusted ISRs (C,I,A)

Handling interrupts is a particular challenge for Bredi: On one hand, interrupt service routines are outside of the TCB. As a result, we need to prevent ISRs from gaining access to register contents of interrupted subsystems or breaking subsystem-ID protection (crucial for **C**, **I**). However, ISRs are critical for **A**: subsystems should not be able to prevent ISRs from executing. Finally, for real-time operation (**A**), *preemptive scheduling* is desirable. We implement the following extensions to cva6 to fulfill these requirements: register stacking, interrupt vectoring, voluntary preemption, non-maskable interrupts.

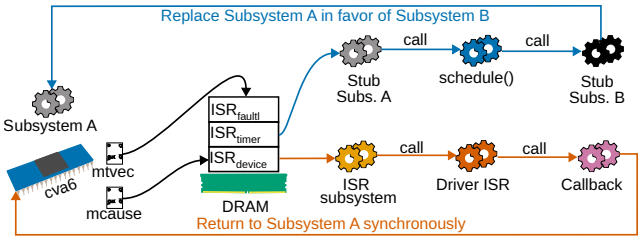


Figure 4: Vectored interrupts in Bredi cva6.

Register Stacking (C,I). We add a second copy of the register set to cva6. For each issued instruction, we track whether it is part of the *interrupt regime*: The interrupt regime begins with the first instruction of the ISR and ends with the return-from-interrupt *mret* instruction. Instructions in the interrupt regime use one copy of the register set, while instructions in the non-interrupt regime use the other. In doing so, registers are never leaked from an interrupted subsystem to an ISR.

In addition to general-purpose and floating-point registers, CSRs are also duplicated for both regimes. As a result, ISRs cannot modify CPU state for the running subsystem. This also prevents ISRs from updating *mepc*, the interrupted address, ensuring that the ISR always returns to the instruction that it interrupted. Hence, ISRs cannot *hijack* execution under the interrupted subsystem's ID. At the same time, we made sure that within one regime, CSRs behave exactly as in the RISC-V specification [58]. In effect, this ISA change only affects few lines of assembly in the interrupt handler.

The MMIO registers in the operations module are also duplicated for both regimes. Accordingly, ISRs can perform capability operations as normal without interfering with the interaction of the interrupted subsystem with the module.

Nesting of interrupts is not necessary for our security model, as interrupt handlers are part of the aTCB: Denial-of-service attacks from interrupt handlers are outside of the threat model, and interrupt handlers are required to terminate in constant time. Furthermore, configurable interrupt prioritization is provided in the platform-level interrupt controller (PLIC) [61]. Hence, we leave nested interrupts for future work.

Interrupt vectoring (C,I). We implement an interrupt vectoring scheme that matches the proposed FastIRQ extension [62], see Figure 4. The CPU maps each interrupt cause to a Northcape capability token, depending on interrupt cause, using a *vector table*. Skadi populates the table with set-subsystem-ID tokens for the interrupt and exception sub-

systems. Consequently, cva6 performs a *subsystem call* into the handler when taking an interrupt, ensuring subsystem-ID isolation between interrupted subsystem and ISR.

Crucially, the subsystem ID in the cva6 MMU is also tracked separately for the interrupt and non-interrupt regime, ensuring immutability of the non-IRQ ID during interrupts.

Voluntary Preemption (A). The timer interrupt is *exempt from register stacking*: taking timer interrupts does not transition cva6 into the IRQ regime. Thus, opposite to device ISRs, we remain on the non-ISR register set, allowing us to *reschedule*. This mechanism is used for preemption in Skadi: Each subsystem receives a write-only capability token that resolves to the vector table entry for the timer interrupt. Subsystems register the address of a *timer stub* in their code segment for the timer interrupt on each subsystem call. Thereby, when a timer interrupt is taken, the timer stub of the current subsystem executes. This timer stub can save and clear registers and perform a subsystem call into the scheduler, which can reschedule and return into any subsystem. Subsystems can disable timer interrupts, making this scheme *opt-in*.

Non-maskable Interrupts (A). Finally, interrupt causes can be set as *non-maskable* using a write-once register. Non-maskable interrupts (NMIs) are always taken, except if the CPU is already executing an NMI. As a result, the timely execution of critical ISRs can be enforced.

4.3 Contribution 3: Custom Instructions (A)

Capability access and subsystem calls are *implicit* operations in Northcape. In other words, capabilities are accessed using ordinary load, store and control flow instructions, akin to virtual addresses. Thus, in contrast to capability systems with explicit capability instructions like CHERI [76] and RV-CURE [42], Northcape does not require custom instructions for **C** and **I**. However, we provide three customizations for improving the *performance* of the system, critical for **A**.

Bredi provides a custom instruction *calls* for subsystem calls, which is a variant of *jalr* that checks the validity of the destination address to be a subsystem call target and triggers an exception otherwise, eliminating one *inspect* operation.

A second custom instruction zeros registers based on a bitmask, saving 18 cycles on a two-way subsystem call.

Finally, we provide custom CSRs for accessing the operations module, eliminating *fences* that would be needed for the MMIO interface and saving tens of cycles per operation.

5 Skadi RTOS Implementation

Our Skadi operating system takes advantage of Bredi in order to maximize the compartmentalization of the operating system. We start with a design overview before detailing three core contributions: Section 5.1 discusses how no run-time software TCB is achieved (critical for confidentiality **C**, integrity **I**), Section 5.2 discusses the lack of a privileged kernel (**C**, **I**), and Section 5.3 discusses performance optimizations for Skadi's subsystem calls (needed for availability **A**).

Skadi is based on the Zephyr [83] real-time operating system. Zephyr is highly modular and configurable: it provides a minimal operating system kernel that implements scheduling and initialization. In addition to that, a large selection of components including a network stack and device drivers can optionally be built. Zephyr is optimized for embedded devices with little memory and low processing power. As a consequence, Zephyr follows the unikernel paradigm, i.e., kernel, optional components and applications are linked into a single binary and execute in a single address space. On boards with hardware support for memory isolation, e.g., an MPU, Zephyr can separate application logic from kernel and subsystems. However, compromise of kernel or subsystems will inevitably cause compromise of the whole system [67].

Skadi takes advantage of Zephyr's highly modular design by introducing mutual run-time isolation of the existing components as *subsystems* using subsystem calls. We further break up the kernel into initialization logic, which mostly becomes part of the Skadi loader, the scheduler, which becomes an unprivileged subsystem, and interrupt handling, which also becomes a subsystem. As a result, Skadi has *no privileged kernel*. We depict exemplary Skadi subsystems and their dependencies in Figure 6 in the appendix.

5.1 Contribution 1: No Run-Time TCB (C,I)

Skadi subsystems are compiled individually into relocatable ELF binaries. To this end, our toolchain allows compiling one or more C modules into a binary. Subsystems can be as small as a single function, which is the case for, e.g., the *random* subsystem. Thus, during the boot process, the subsystems need to be *relocated* into their own protected address space. Hence, we provide a minimal privileged firmware: the *Skadi Loader*. The loader is conceptually part of the *Bredi SoC* and is assumed to be *physically protected* in an immutable ROM.

After system reset, the Skadi loader starts execution at subsystem ID 0. At this point, only Northcape's *root capability* exists, which comprises the entire physical address space and is encoded such that type, ID and nonce are zero. In effect, the loader can use it indistinguishably with ordinary 32-bit physical addresses initially. The loader adds a subsystem-ID bound restriction of 0 to the root capability to make it inaccessible to subsystems and DMA devices.

The loader then proceeds to *relocate* subsystems: It first allocates capabilities for code, data and bss segments using *create* operations as outlined in Section 2.3. The loader proceeds to decompress the LZ4 [25]-compressed ELF segments into said capabilities. This is followed by a standard ELF run-time linking and relocation process: We compile subsystems using RISC-V's *large code model* [23], where code and data

symbols are referenced via relocation tables co-located with each function. Relocation table entries are 64 bits wide and natively fit Northcape tokens. Hence, subsystems can import and export data and subsystem calls as simple ELF symbols, which the loader can resolve via standard linker symbol tables. This allows us to use a standard compiler and linker, requiring no modifications to the toolchain. In contrast, RV-CURE and CHERI require custom compiler support [42, 76].

Subsystems can also import *MMIO mappings* by importing an ELF symbol that encodes the physical address and length of the MMIO region, for which the loader creates a capability accordingly. We maintain compatibility with Zephyr's build-time device tree parsing by declaring the imported MMIO regions as *functions* and using their *address*.

Subsystems can register *initialization functions* to perform setup work. To this end, the loader will assign each subsystem a monotonic subsystem ID from Northcape's 32-bit subsystem ID space. It will then create a set-subsystem-ID capability for that subsystem ID, resolving to a simple *initialization trampoline*. It will also apply *subsystem-ID bound restrictions* to the capabilities holding the subsystem's code and data, ensuring they cannot be used by other subsystems. The loader can perform a *subsystem call* into the initialization trampoline, ensuring mutual isolation from the subsystem. The trampoline then jumps into the code segment of the subsystem, which begins execution at its assigned subsystem ID, but is unable to compromise the loader or any other subsystem.

As soon as all subsystems are loaded and initialized, the loader will destroy itself by transitioning into the *trapdoor* subsystem. The *trapdoor* subsystem performs *revoke* on what remains of the root capability. This invalidates all set-subsystem-ID capabilities with subsystem ID 0, making it impossible to re-gain execution at that subsystem ID. For that reason, all secrets accessible to the loader—such as direct capabilities for subsystem code and data—become inaccessible. The *trapdoor* then performs a subsystem call into the *scheduler* without valid return address. From that point on, only *unprivileged* subsystems with non-zero ID remain. As a result, Skadi manages to achieve *no run-time software TCB*.

A drawback of our approach is that Skadi can only support static configurations. Loading subsystems at run time is not possible. The same is true for standard configurations of Zephyr, which Skadi is based on. Zephyr provides the LLEXT extension mechanism that allows run-time loading of additional components. However, LLEXT is only intended for userspace components and rarely needed. In the future, Skadi could be extended with untrusted loading similar to Sancus [56] to resolve this limitation.

5.2 Contribution 2: No OS Kernel (C,I)

Skadi has no kernel. Instead, all kernel functionality is realized via compartmentalized *subsystems* that cooperate using *subsystem calls*. Crucially, subsystem calls require no trusted intermediary like the *Switcher* in CHERIoT RTOS [10].

A *scheduler* subsystem maintains thread control blocks and implements scheduling policies. This is also where *cancellation points* such as *k_fifo_get* are implemented. Waiting at a cancellation point is done via a *subsystem call*.

The *voluntary preemption* mechanism explained in Section 4.2 allows us to implement *preemptive scheduling*. Remember that each subsystem call updates the handler for the *timer interrupt*. Thus, each subsystem can provide a *timer stub* in its own code segment and ensure it is registered whenever the timer interrupt is enabled. This timer stub can then perform a subsystem call into the yield endpoint, which can decide whether the system should reschedule. In case the active thread changes, the scheduler can perform a normal *context switch* on its own stack and then simply return to a different subsystem. Otherwise, it returns to the timer stub.

Device interrupts and exceptions are handled via separate subsystems. The loader registers the correct subsystems for each interrupt cause. The *fault* subsystem is responsible for safely halting the system when encountering an exception. It has no other function and is registered only for exceptions. While *fault* currently handles exceptions globally, per-compartment interrupt handlers like in CHERIoT RTOS are possible [10]. To this end, a vectoring mechanism for exceptions as well as an equivalent of our voluntary preemption mechanism could be devised. The *ISR* subsystem contains the CPU-specific first-level interrupt service routine. On Bredi, it will defer interrupts directly to the second-level interrupt controller (PLIC) via a subsystem call, which will call the interrupting device driver's ISR.

Typical library functionality is provided either *inline* or as a subsystem: The *libc* subsystem implements standard C library functions like *printf*. Basic string functions like *memset* can optionally be inlined into each subsystem for increasing performance. We also implement an untrusted allocator subsystem following the strategy introduced in Section 2.3.

Finally, all drivers are subsystems. Drivers can export device structs akin to Zephyr's device model [84]. The loader will derive corresponding capabilities and make them available to other subsystems, which can use them to call the device API via subsystem calls. Drivers can register with the PLIC subsystem for servicing IRQs. Finally, DMA drivers can *delegate* capabilities to devices akin to the strategy in Section 2.3. Crucially, DMA devices can *only* use capabilities explicitly delegated to them. Thereby, Skadi provides byte-granular spatial and temporal security for DMA devices.

5.3 Contribution 3: Calling Convention (A)

Overview. Our subsystem calling convention is detailed in Figure 5: The caller allocates a return trampoline (1), zeros all registers to prevent information leakage (2) and jumps into the *callee trampoline* (3). The callee trampoline allocates a stack (4) and sets the timer stub (5). After that, the C function is

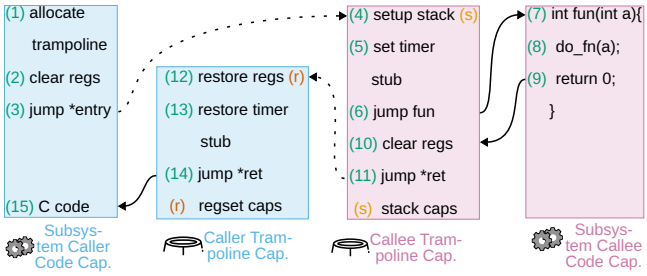


Figure 5: Full subsystem call. Solid lines denote control flow, whereas dashed lines denote subsystem calls.

invoked (6-9). Finally, the callee trampoline zeros its registers (10) and returns (11). The caller trampoline then restores the register set and timer stub (12-13) before returning to the C code (14-15). Crucially, code and data of the trampolines are part of each subsystems' code and data segment, which is why they require *no trusted intermediary*. The used set-subsystem ID capabilities are created by the Skadi loader during relocation.

Stack Allocation. A core detail of our subsystem call convention is the *allocation* of per-subsystem register sets and stacks. Each Skadi subsystem allocates a build-time-configurable number of stack frames and register sets into a pool. Allocation uses either an atomic bitmap or a uniprocessor per-IRQ-regime bitmap, offering a size-performance tradeoff while ensuring SMP and IRQ safety. Allocation uses the $O(1)$ *ctz* (count trailing zeros) instruction. If stack frames are available, we can allocate one in a single cycle. Section 7.4 shows that this facilitates fast subsystem calls. If all stack frames are occupied, we can spin or trigger an exception.

Previously proposed subsystem calling conventions use a per-thread stack instead, and share it between caller and callee. While reducing memory usage compared to Skadi, this necessitates clearing the stack on return [10, 68] or relying on *uninitialized capabilities* [36, 80], incurring run-time cost.

Control Flow Protection. We incorporate control flow protection for both the caller and the callee. Callees can only be called by other subsystems via callee trampolines that they expose explicitly. Callee trampolines are detected by the loader using a naming convention. The loader creates *set-subsystem ID* capabilities for each callee trampoline. In order to be able to use such a capability, the caller needs to import it during relocation or request it from the loader via an API.

Callee trampolines are considered public, i.e., the loader will provide the corresponding tokens to any caller. Access control to specific callee trampolines by the loader would be possible. To this end, a new variant of the *set-subsystem ID* restriction that applies *subsystem-ID bound* to the caller could be added. However, subsystems can act as intentional or unintentional *deputies* that allow *transitive* calls of other subsystems. Hence, no such solution has been implemented.

Control flow protection for the caller comprises ensuring that the callee can only return to the correct caller trampoline, and that such a call can only happen once. To ensure a return to the correct code, in step (11), we require a *return capability*. The return capability has *set-subsystem ID* protection with the subsystem ID of the caller. The return capability is passed in the *ra* (return address) register, following the standard RISC-V convention. The callee can perform a subsystem call using *calls* on *ra* to return. Hence, no return to an *arbitrary* code offset is possible due to the *subsystem-ID bound* restriction of the caller's text segment.

Callee trampolines can use *inspect* to identify the subsystem that created the return capability. However, subsystems can provide return capabilities collected from other callers. Hence, the caller cannot be reliably identified currently. Future work can add an unforgeable *caller identity* in a register.

Skadi does not enforce a global *sequence* of calls and returns. Callees can perform any subsystem call and attempt to return to any subsystem for which they have a return capability, and are not required to return to their caller immediately. Subsystems must be able to handle such concurrent calls.

Preventing replay attacks against the caller depends on the caller trampoline (1). The trampoline has a *valid* flag. The flag is set during allocation and unset in the caller trampoline. If the caller trampoline is *invalid*, its code jumps into an *error handler* and refuses returning into C code with a possibly invalid stack or register set (15). We use the same synchronization as for stacks to prevent race conditions.

Trampoline Generation. Trampolines are generated using function-like macros from call name and signature. Porting of Zephyr components merely requires adding wrapper macros for subsystem calls. Using `_Generic`, we distinguish void trampolines and clear return registers accordingly.

We *deduplicate* and *prune* the caller trampolines, reducing *code size*: caller trampoline macros generate an invocation of an external function whose name encodes the call name and signature. At build time, for each subsystem, we generate a source file that implements the caller trampolines according to the names of unresolved functions that match the schema.

6 Security Discussion

Skadi and Bredi can address the attack vectors that we motivated at the beginning of this work—*application software*, *operating system kernels*, and *peripheral devices*. The key idea to solve this challenge is to combine *untrusted operating system* approaches from capability literature [9, 10, 33] with backwards-compatible enforcement of capabilities at the system bus level. Thereby, we incorporate security against untrusted DMA devices [2, 51]. Table 1 lists the detailed security guarantees provided by Skadi and Bredi and their key implementation mechanisms. In a nutshell, the strong isolation of Skadi components facilitates confidentiality (**C**) and integrity (**I**) for the entire *sphere of protection* of the subsystem. As the loader, the only trusted software component,

Table 1: Summary of specific security guarantees provided and their key mechanism in hardware and software.

Goal	Bredi	Skadi
Spatial Memory Isolation (**C**,**I**)	subsystem ID protection and restrictions, nonce, monotonicity of operations (Section 2.2)	Private code and data, no run-time software TCB, no kernel (Section 5.1, Section 5.2)
Temporal Memory Isolation (**C**,**I**)	*lock* exclusive access, tracking of parent-child relationships and recursive resolution (Section 2.2)	no run-time software TCB, no kernel, untrusted scheduler and allocator (Section 5.1, Section 5.2)
DMA isolation (**C**,**I**)	AXI MMU (Section 4)	untrusted DMA driver (Section 2.3, Section 5.2)
ISR isolation (**C**,**I**)	register stacking, interrupt vectoring (Section 4.2)	ISR table managed by loader (Section 5.2)
Preemption (**A**)	voluntary preemption hardware support (Section 4.2)	timer stub in trampolines (Section 5.3)
NMIs (**A**)	NMI hardware support (Section 4.2)	ISR subsystem (Section 5.2)
Fast operations (**A**)	cache hierarchy (Section 4.1), instructions (Section 4.3)	subsystem calling convention (Section 5.3)

is destroyed after load time, Skadi accomplishes this *without run-time software TCB*. The loader is also small and can be manually or automatically verified. Using debug information, we counted 2150 LoC specific to Skadi and ELF relocation are compiled into the loader binary for a complex scenario. Including board initialization and standard libraries, the total loader comprises 8610 LoC. A similar trusted loader is used, e.g., by CheriTrEE [70] for the same purpose. As Skadi's *compartmentalization* is crucial for achieving **C** and **I** without compromising availability (**A**), in the remainder of this section, we argue why compartmentalization cannot be broken.

In order to protect **C** and **I** of subsystems' private code and data, we relocate subsystems' ELF segments into subsystem-id-bound capabilities. Guessing attacks on these capabilities are prevented by *subsystem-ID bound restrictions*: attackers need to both *guess* the nonce and *hijack* the subsystem ID. Our subsystem call trampolines are designed to prevent being able to change the subsystem ID without transferring control on both call and return. Without using subsystem calls or control over ID 0, there is no way to change the subsystem ID, preventing hijacking. Due to *register stacking* and *IRQ regimes*, ISRs cannot interfere with interrupted subsystems.

Dynamically allocated memory is protected by the unforgeability of the capability tokens and optionally exclusive access via *lock* or subsystem ID bound restrictions. Not adding subsystem restrictions to allocated memory by default allows sharing the memory with other subsystems or devices. The allocator cannot use *revoke* to break **C** or **I**, as *revoke* overwrites the data and invalidates overlapping capabilities.

Finally, **A** relies on NMIs. Interrupt handlers are not allowed to perform denial-of-service attacks, as they are in the aTCB. Subsystem calls between ISR subsystems were designed to be real-time capable. Adversaries outside of the ISR subsystems cannot prevent them from executing, as interrupts can be made unmaskable. Thus, the IRQ mask is *write-once*.

7 Performance Evaluation

Our evaluation captures the performance overhead imposed by Bredi and Skadi. We implement our prototype on a Digilent Genesys 2 FPGA board. The L1 and L2 NTLBs have sizes 16 (instruction) / 32 (data) and 512, respectively. The L1 NTLB is fully associative, while the L2 NTLB uses 8-times associativity. Our benchmarks cover chip area, code size, compute and networking performance, and real-time suitability. We compare Zephyr and Skadi on Bredi.

Furthermore, where possible, we try to evaluate equivalent scenarios on Linux v5.10.7. *This is an apples-to-oranges comparison, as many APIs and system services differ significantly between Zephyr and Linux.* We chose to provide this comparison anyway: The comparison with Linux shows that Skadi's performance is within the same order of magnitude as a commonly used general-purpose operating system, confirming our claims about Skadi's *real-world practicality*.

7.1 Hardware: Chip Area, Frequency and Power

We synthesize Bredi using Xilinx Vivado 2024.2 [6] at 50 MHz akin to the original cva6 design [82]. In Table 2, we report the number of FPGA primitives required for the components of Bredi as well as Bredi-cva6 and the original cva6. Our design meets all timing requirements.

Table 3 includes ASIC synthesis results for the Bredi components. We utilize Yosys (0.64, cva6, 0.47, others) [79] in conjunction with the OpenLane2 RTL-to-GDS-flow [31] and the OpenRAM memory generator [37], targeting Sky130 [35]. We report the area cost of both our design and the utilized macros and the f_{max}. For cva6, a different cache subsystem (Write-through, WT) than in the FPGA results was used due to toolchain issues with the high-performance D-cache. Also, antenna optimizations had to be disabled in cva6.

Vivado reports a power consumption of 2.739 W and 2.362 W for our SoC with and without the Bredi components, respectively. Both power consumptions are dominated by I/O, clock generation and device static (around 1.93 W for both). The majority of the increase is caused by signals (Δ 0.151 W) and block RAMs (Δ 0.112 W).

Our chip area and frequency results indicate that inclusion of Northcape into an FPGA or ASIC design is practical. The L2 NTLB has the highest chip area, which is caused by the complex hazard avoidance logic and high associativity.

Table 2: Area in lookup tables (LUTs), flip flops (FFs), multiplexers (MUXs), and Block RAM (BRAMs) for Bredi and cva6.

Component	LUTs (%)	FFs (%)	F7 MUXs (%)	F8 MUXs (%)	BRAMs (%)
AXI MMU	4 085 (2.00)	832 (0.20)	2 (0.01)	0 (0.00)	0 (0.00)
Resolver	1 640 (0.80)	506 (0.12)	58 (0.06)	0 (0.00)	0 (0.00)
Operations	9 280 (4.55)	4 675 (1.15)	251 (0.25)	0 (0.00)	2 (0.45)
L2 NTLB	20 830 (10.22)	2 230 (0.55)	357 (0.35)	57 (0.11)	70 (15.73)
Bredi cva6	63 245 (31.03)	39 971 (9.81)	4 367 (4.29)	1 580 (3.10)	28 (6.29)
Original cva6	46 764 (22.95)	25 817 (6.33)	2 060 (2.02)	426 (0.84)	28 (6.29)

Table 3: ASIC area of components and memory macros in μm^2 and f_{max} in MHz for Bredi and baseline cva6.

Component	Design μm^2	Macro μm^2	f_{max}
AXI MMU	215 546.73	-	57.56
Resolver	190 518.97	-	63.28
Operations	376 019.39	278 521.33	46.98
L2 NTLB	1 174 691.63	9 928 473.92	44.64
Bredi cva6 (WT)	4 331 868.35	8 582 467.68	11.76
Original cva6 (WT)	2 628 902.58	8 582 467.68	20.94

The ASIC results show that Bredi components besides the NTLB L2 are an order of magnitude smaller than the baseline cva6 CPU. The size and associativity of the NTLB L2 can be reduced to reduce its area cost. Finally, the limited f_{max} of Bredi cva6 is caused by limitations of the OpenRoad placer.

Implementing Northcape operations in a hardware FSM is necessary for our security goal of *no run-time software TCB*. If this is not required, capability operations could be implemented by privileged firmware on a root-of-trust CPU [14].

7.2 Software: Code Size

Table 4: Code size in byte for Skadi and Zephyr.

Category	Hello World	HTTP	zperf
Disk size	739 274	2 436 060	3 110 196
	49 340	557 272	842 488
Text	405 694	1 465 942	3 792 664
	26 228	175 066	389 332
Data	14 796	56 852	79 852
	9604	54 886	94 606
R/O data	71 536	149 572	237 056
	-	-	-
BSS	103 192	1 935 060	1 918 092
	13 501	327 274	358 527
Subsystems	12	21	23
Callee Trampolines	219	450	490
Caller Trampolines	50	222	235
Stacks / Register Sets	4/4	16/16	12/12
ELF Overhead	1 872 575	4 606 820	6 139 418
Total memory	1 774 696	7 258 552	10 290 648

We detail the code size for three configurations of Skadi in Table 4: a simple hello-world example, a basic socket-based HTTP server, and zperf (Zephyr's implementation of iperf).

We report total binary size (*disk size*) and the size of the text, data, read-only data (rodata) and bss segments. A large part of the binary also comprises metadata of subsystems—relocation sections, symbol tables etc., that are reclaimed after the load stage. We provide the sum of the sizes of these metadata *before LZ4 compression* in the *ELF Overhead* row. Finally, for Skadi, we provide the total allocated memory after initialization. Zephyr solely uses the .bss segment.

Our code size evaluation shows that the overhead of Skadi's compartmentalization is significant. As we explained in Section 5, Zephyr is a *unikernel* operating system: In the default configuration, all operating system components live in the same address space. On the other hand, Skadi's components are *relocated at run time*: Thus, all components need to carry additional metadata such as relocation entries, explaining the increase in disk size. The increase in run-time memory use can be attributed to subsystem call trampolines, per-subsystem stacks and inlining of some library functions.

While Skadi has a high size overhead, it is still small enough for most modern application scenarios. In particular, nowadays, the amount of memory on embedded systems is mostly limited by *power budget* rather than *cost*, with advancements in energy efficiency being made continuously [16]. Thus, future embedded devices will likely comprise larger memories, making this drawback irrelevant in practice.

7.3 System: Load Time

Table 5: Load times in s for Skadi and Zephyr.

Category	Hello World	HTTP	zperf
Skadi	14.83 ± 0.01	20.96 ± 0.01	27.74 ± 0.00
Zephyr	0.01 ± 0.00	0.18 ± 0.02	0.23 ± 0.01

We provide load times over five repetitions for configurations from Section 7.2 in Table 5. The results reveal a significant overhead for Skadi, caused by dynamic relocation.

We specifically designed Skadi around subsystems in *binary* format, such that subsystems from different vendors can be integrated without having to reveal source code. Hence, dynamic relocation is a necessary overhead.

Table 6: Microbenchmarks of Northcape operations.

Benchmark	CPU Cycles	Ops Cycles
create	629.27 ± 4.39	11.00 ± 0.00
derive	621.97 ± 6.61	11.00 ± 0.00
drop	255.88 ± 4.50	12.00 ± 0.00
unlock	124.81 ± 1.29	17.00 ± 0.00
merge	407.13 ± 3.20	16.00 ± 0.00
clone	322.35 ± 5.30	14.00 ± 0.00
revoke (1 MiB)	39 894.21 ± 59.62	34 934.54 ± 5.90
lock	402.68 ± 5.76	16.00 ± 0.00
inspect	284.28 ± 2.86	8.00 ± 0.00
restrict	382.45 ± 3.96	10.00 ± 0.65

Table 7: Microbenchmarks of Northcape subsystem calls compared against legacy Zephyr system calls via PMP.

Benchmark	CPU Cycles	Instructions
syscall (one-way)	364.16 ± 2.76	140 ± 0.00
syscall (two-way)	485.26 ± 2.75	196 ± 0.00
subs. call (one-way)	250.77 ± 12.26	113 ± 0.00
subs. call (two-way)	402.56 ± 13.43	180 ± 0.00

However, due to toolchain limitations, subsystems are provided as ELF object files. A different file format, e.g., shared libraries, could provide faster relocation in the future.

7.4 System: Microbenchmarks

We provide a microbenchmark of Skadi operations in Table 6. We record the total duration of the corresponding driver function and the number of cycles used by the operations module.

For the subsystem call microbenchmark in Table 7, we measure one-way and two-way duration of subsystem calls in comparison with Zephyr PMP system calls.

All in all, all operations except *revoke* complete in under 1,000 cycles including driver overhead (mainly packing arguments into registers). For *revoke*, the bottleneck is zeroing out the segment in DRAM, a security necessity. However, *revoke* is only used on rare occasions, such as at a few boot steps, making the overhead irrelevant in practice.

One-way and two-way subsystem calls take appx. 251 and 403 cycles and outperform legacy systemcalls at 364 and 485 cycles, respectively. This is caused by the lower number of retired instructions for Skadi, courtesy of our custom instructions (Section 4.3). Remember that subsystem calls are essential for *mutual isolation*, while system calls only isolate *the privileged kernel from the userspace, not vice versa*.

Still, subsystem calls are not free: The caller trampoline needs to save and restore the *entire* callee-saved set and perform register set and stack allocation. Function calls can rely on the existing stack pointer and only need to save and restore callee-saved registers that they modify, allowing them to be much faster. In fact, RISC-V functions that solely operate on registers can have an overhead as low as two instructions (a call and a return) [58]. Thereby, subsystem calls will inevitably cause a performance overhead.

Table 8: Compute benchmarks for Skadi, Zephyr and Linux.

Benchmark	Linux	Zephyr	Skadi	Δ
Coremark/sec	92.10	110.02	110.14	0.13
Stream Copy sec	11.23	6.04	6.00	−0.04
Stream Scale sec	12.75	6.66	6.65	−0.01
Stream Add sec	17.53	9.03	9.02	−0.02
Stream Triad sec	18.62	9.50	9.50	0.00

7.5 System: Compute Performance

We utilize eembc coremark [30] and Stream [52] for compute and memory performance. Results can be found in Table 8. Extended results are available in our artifact [1]. Overall, the differences between Skadi and Zephyr on Bredi are negligible. This can be explained by a 100% hit rate on the L1 NTLB, which does not impose any latency on cva6' pipeline, and a lack of subsystem calls during the benchmark.

Linux' comparatively lower performance can be explained by TLB misses. We use cva6' default setting of 16 instruction and data TLBs, respectively, which are shared between kernel and user space. *We do not claim that this result is generalizable.* We solely provide the value to point out that Zephyr and Skadi perform in the same order of magnitude as Linux.

7.6 System: Network Stack Performance

We measure ICMP echo round-trip time using *ping*, TCP throughput using *iperf* and *zperf*, network stack latency using hardware timestamps, and an MQTT scenario similar to CHERIoT [9]. For the network stack latency, we use *hardware timestamps* collected by a ha1588 hardware time stamp unit (TSU) to determine reception and transmission times at the MAC-PHY-interface. We then determine the delay between TX or RX time at the network socket, relative to ha1588's time. On Linux, RX and TX delays are measured via *standard software timestamps*, causing a slight undercount. Our MQTT scenario is based on Zephyr's MQTT publisher sample with TLS enabled, measuring the time to publish three messages over 100 iterations. *MQTT* is omitted on Linux, as the MQTT implementation is Zephyr-specific.

We report the most important results in Table 9. Full results are given in our artifact [1]. The performance overhead that Skadi and Northcape impose on our ICMP and zperf scenarios are significant. In particular, we measure an appx. 3x-4x overhead between the baseline and Skadi scenarios. We investigated the sources of the encountered overhead more closely for the zperf sample. We found that over the 60 seconds of benchmarking, operations took approximately 2.40 CPU seconds alone, and subsystem calls took 0.62 seconds. Opera-

Table 9: Networking benchmarks for Skadi, Zephyr and Linux (omitted for MQTT). Number of subsystem calls (total for ping and iperf, otherwise per-iteration) provided for context.

Benchmark	Avg±std.dev.		Δ
Ping latency (ms)	2.56 ±	0.08	
	(Zephyr) 0.88 ±	0.02	x 2.92
	(Linux) 1.92 ±	0.15	x 1.33
Subsystem calls	59788 ±	-	
iperf TCP (kbps)	1499.33 ±	2.52	
	(Zephyr) 4396.67 ±	6.81	x 0.34
	(Linux) 9031.17 ±	1733.20	x 0.17
Subsystem calls	1051756 ±	-	
RX duration (ns)	2 138 493.63 ±	13 707.45	
	(Zephyr) 534 142.81 ±	8120.03	x 4.00
	(Linux) 1 346 524.21 ±	136 558.72	x 1.59
Subsystem calls	290.05 ±	11.43	
TX duration (ns)	1 516 145.02 ±	11 670.62	
	(Zephyr) 530 697.96 ±	13 202.11	x 2.86
	(Linux) 1 050 532.63 ±	203 162.69	x 1.44
Subsystem calls	235.08 ±	2.43	
MQTT-TLS (ns)	2 057 174 000.84 ±	5 772 516.87	
	1 993 484 845.47 ±	30 590.98	x 1.03
Subsystem calls	2058.80 ±	147.17	

tions and subsystem calls can be reduced by more coarse-grained isolation, at the expense of a larger impact of compromise, a common tradeoff in compartmentalization [44]. Optimization of the caching hierarchy and the toolchain are also possible. We encountered 8 168 365/9 207 016 (instruction/data) misses and 319 982/35 186 (resolver/ operations) misses in the NTLB L1 and L2, respectively.

However, we believe that the performance in absolute numbers still shows that Skadi is sufficiently fast for most embedded applications. In particular, all measurements except TCP throughput have an overhead less than factor two in comparison to Linux. As Zephyr's TCP throughput is lower than Linux' as well, we assume that Zephyr's network stack just provides lower performance on our platform in general.

We see two avenues for future optimization. First, our isolated compartments conceptually match Zephyr's subsystems as closely as possible. This is the most fine-grained compartmentalization of the system, whereas in practical scenarios, some compartments could potentially be merged into one, reducing subsystem calls. This is done in Linux [49] for the Ethernet driver and DMA driver, for example. Second, it might be possible to eliminate subsystem calls between the Ethernet and DMA drivers, networking subsystem and scheduler at the expense of breaking existing APIs and driver models.

7.7 System: Real-Time Capability

For evaluating the real-time capability of Skadi, we cover a middle ground of suitable evaluation approaches in the literature: We cover Zephyr's scheduler microbenchmark [34] as well as periodic interrupt response time and maximum interrupt rate without missed interrupts [13].

Table 10: IRQ benchmarks results for Skadi and Zephyr.

Benchmark	Avg±std.dev. (ns)		Δ
independent	68 423.16 ±	1529.02	
	24 127.16 ± 32	149.03	x 2.84
Subsystem calls	1144.68 ±	18.55	
monotonic period	136 000		
	70 000		x 1.94
monotonic	58 826.32 ± 18	714.18	
	8392.42 ± 12	120.04	x 7.01
Subsystem calls	7.02 ±	0.14	

In order to test our availability guarantees, on Skadi, we make our external interrupt *non-maskable* and *disable* interrupts while busy-waiting. The Zephyr scenarios run with interrupts enabled, as they would otherwise not complete.

Results of our IRQ experiments can be found in Table 10. Extended results and our scheduler microbenchmark can be found in our artifact [1].

All in all, Skadi incurs an overhead both in terms of constant latency and in latency range and standard deviation. For the IRQ scenarios, Skadi's overhead is between 3 and 7 times. Similar to the networking overhead determined in Section 7.6, this is caused by compartmentalization (see the number of subsystem calls needed). However, in absolute numbers, Skadi is still able to respond to interrupts in roughly $\frac{1}{10}$ of a millisecond, with a comparable standard deviation.

The results for the scheduler benchmark indicate that Skadi's strong compartmentalization imposes an overhead mostly between 3 and 10 times. One notable outlier is thread cancellation—if this occurs, Skadi needs to invoke a callback in each subsystem to release any stacks that might still be associated with this thread. Again, considering especially the absolute numbers, we believe that the results indicate that Skadi is suitable as a soft real-time system.

8 Related Work

We cover three areas of related work: First, we discuss the differences to the previously published Northcape concept [2]. Then, we compare Bredi and Skadi with related capability architectures and finally non-capability approaches.

8.1 Improvements to Northcape

Ackermann et al. [2] proposed Northcape as a purely theoretical capability system without hardware implementation and operating system. Additionally, Bredi's realization of Northcape improves the concept in three key areas.

Preventing guessing of capability tokens and subsystem identifiers. Northcape originally used a *secret* subsystem identifier in an *architectural register tid* (task ID). Each *locked*

capability would then only be accessible for the subsystem that locked it, using *tid* to distinguish subsystems. Thereby, exclusive access was always *per-subsystem*. This mechanism has a crucial drawback: *the tid can be leaked or guessed*, allowing circumvention of exclusive access. We removed the *tid* register and re-designed both locking and restrictions: They are now independent mechanisms. Hence, *lock-holder tokens* and *subsystem-id restrictions* are our contribution.

Extending the set of access permissions. In addition to the capability access permissions *read*, *write*, *execute*, *lockable*, Bredi also adds *irq_accessible*, *cacheable_data* and *cacheable_tlb*. *irq_accessible* controls whether capabilities can be accessed in ISRs, which is used, e.g., for preventing overwriting the timer stub from ISRs, which would break ISR isolation. The cache permissions can be used for hard real-time processing in specific subsystems.

Implicit subsystem calls. Finally, Bredi makes two changes to the *calls* operation. First, subsystem calls in Bredi can be accomplished using regular jump instructions. To this end, the cva6 MMU ensures mutual isolation *irregardless* of the instruction that commences a subsystem call: Every attempt to subvert control flow by jumping over the first instruction or reading secret data from the set-subsystem ID capability is met with an exception. *calls* solely exists to optimize performance. Second, Bredi's *calls* does not return a context pointer—instead, the context is saved in the set-subsystem-ID capability in Skadi. This increases software flexibility.

8.2 Capability Systems

Most capability systems do not consider attacks from devices (*DMA attacks*) in their core concept (some simply because they precede DMA [19, 47, 75]). Capability systems such as CAPSTONE [80] and CHERI [76] are focused on protecting *software* from *system-level software adversaries*. Several related capability systems also focus solely on *in-process compartmentalization*, i.e., isolating mutually distrusting components or libraries within a process address space in a trusted kernel setting [28, 42, 46, 74].

CHERI was recently extended to support IOMMU-like functionality [22]. The proposed CapChecker is initialized with a set of CHERI capabilities, which are identified by a DMA device by a number. DMA transactions are then interpreted relative to the capability, enforcing spatial memory safety. Crucially, this work relies on trusted drivers, imposes a hard limit on the number of capabilities a device can use and requires hardware changes to the DMA device for full security. Bredi and the underlying Northcape do not have these limitations, provide spatial and temporal memory safety, and allow both CPUs and devices to create capabilities.

Userspace drivers have also been proposed for CHERI platforms, using capabilities to access a safe subset of device MMIO without system call [29, 54]. However, this design requires a *trusted* kernel: A kernel-level driver needs to setup an IOMMU or Graphics Translation Table (i.e., the MMU of a GPU) to prevent the driver from abusing the device as a *confused deputy*. These mechanisms operate at *page* granularity, failing to prevent attacks like Thunderclap [50] in principle.

Furthermore, CHERIoT RTOS [10] resembles Skadi, comprising a fully privileged boot-time-only loader that bootstraps unprivileged compartments. However, in CHERIoT RTOS, switcher and allocator remain part of the run-time TCB [10]. The dIPC system [72] similarly proposes direct inter-process communication for the CODOMs architecture [71] using trampolines and a trusted intermediary, akin to CHERIoT RTOS. Furthermore, other untrusted operating systems in the CHERI domain rely on a privileged run-time nanokernel [33] or do not consider DMA devices in the threat model [8, 10].

Related work on the M^3 operating system [14, 38, 39, 59] takes a different approach: A computer system can be disaggregated into isolated *tiles* that hold CPUs or accelerators. The M^3 kernel can then facilitate interaction between tiles or between tiles and the system memory using a *distributed* capability system. This is enforced by a per-tile access control component similar to the Bredi AXI MMU. Crucially, like in other microkernel operating system, the M^3 microkernel forms a run-time software TCB. Also, there is no support for local *subsystem calls* with mutual isolation, as processing cores use legacy memory protection like MMUs or PMPs.

Bastion [60] implements a similar architecture to M^3, but relies on custom firmware instead of M^3 and uses access control lists (ACLs) for memory access control. The authors further propose a token capability solution as future work.

8.3 Non-Capability Approaches

In contrast to *securely containing* vulnerabilities in kernel software, one can also *strive to prevent* exploitable vulnerabilities in kernel code. Microkernel operating systems do just that: they move all parts of the kernel that do not strictly require operating with elevated hardware privileges into userspace, reducing the attack surface of the overall system [18, 32, 41]. High-performance IPC is then used for communication between the compartments. SeL4 [43] combines this approach with formal verification of the microkernel, formally proving the absence of vulnerabilities in the TCB. We consider SeL4 to be an orthogonal approach to the reduction of a run-time TCB: SeL4 aims at preventing exploitation of the software TCB, at the expense that changes to the TCB require costly re-verification [32]. Skadi aims at eliminating the TCB, at the expense of performance, chip area and code size. Both approaches could be combined in future work, e.g., by formally verifying the Skadi loader and Bredi hardware TCB.

Confidential computing also strives to reduce the software TCB. Commodity implementations of this technology are ARM CCA [12], Intel TDX [24] and AMD SEV [7]. CHERI-TrEE, a system for confidential computing on the CHERI architecture, has also been presented [70]. For CCA, various ap-

proaches that provide security in the presence of compromised devices, operating systems and application software have been proposed [17, 40, 73, 85]. In particular, HiveTEE [40] promises to isolate an arbitrary number of secure application compartments from untrusted devices, operating systems and hypervisors, similar to Skadi's security model. However, as confidential computing technologies rely on a run-time trusted software *Monitor* for configuration of hardware security primitives and transition between compartments, vulnerabilities in trusted software components continue to jeopardize the security of the overall system [65]. Furthermore, the underlying hardware protection mechanisms impose a limit on the achievable granularity (e.g., page or 16-byte MTE memory region for HiveTEE). CHERI-TrEE provides byte-granular memory protection and subsystem calls courtesy of CHERI, but relies on a trusted software interrupt handler as its run-time TCB and does not consider untrusted devices [70].

The ongoing advances in confidential computing show that even in MMU-based systems, the necessary TCB can also be minimized. However, using capability-based memory access control has two advantages: Capabilities support *delegation* and *unprivileged modification using operations*, allowing components to share memory efficiently and securely without a trusted intermediary. In turn, this facilitates *subsystem calls*, the key building block for mutual separation of distrusting components in Skadi. Also, capabilities provide *byte-granular* security while MMUs work with larger *pages*. Overall, Skadi and Bredi can protect any software at *function granularity* with *mutual distrust* and *byte granularity*.

However, physical attacks like bus snooping or cold boot are out of scope for Bredi, while SGX, CCA, SEV and TDX can provide security against such attacks. Future extensions to Bredi that address this shortcoming are possible.

Finally, Skadi provides solely a *mechanism* for compartmentalization. Hence, of the properties defined by Lefeuvre et al. [45], we are able to provide *security and safety* and limited source code *compatibility* with legacy Zephyr modules. Crucially, the focus of our work was providing *security without run-time TCB*, which none of the works surveyed by Lefeuvre et al. achieves. Skadi could utilize automatic *compartmentalization policies* to improve *usability* [45].

9 Conclusion and Future Work

We have presented the Skadi operating system and Bredi SoC, the first implementation of the Northcape capability architecture with significant conceptual improvements. In combination, Skadi and Bredi achieve security in the presence of *malicious application software, operating system kernels and peripheral devices* for embedded devices *without run-time software TCB* and *with soft real-time suitability*.

Within the scope of this work, we focused primarily on the *security* of our prototype. *Performance* could be improved further by adding custom compiler optimizations or using a different programming model that avoids domain switches [64].

Bredi and Skadi are a promising foundation for future research. The high security provided by Skadi and Bredi in conjunction with their real-time operation make them suitable as enabling technology for, e.g., high-security network infrastructure devices like 5G base stations. The *hardware-backed subsystem ID* in conjunction with *restrictions* are also a very powerful primitive for providing additional security guarantees to system peripherals on a *per-capability* basis.

Acknowledgments

We would like to thank the anonymous reviewers for their critical comments and questions as well as our anonymous shepherd for their guidance during revision. We thank Dr. Michael Schwarz for his valuable feedback on our manuscript.

Our work was funded by zukunft.niedersachsen, the joint science funding program of the Lower Saxony Ministry of Science and Culture and the Volkswagen Foundation.

Ethical Considerations

Stakeholders and Scope of Analysis. We exclusively target *defensive* applications: All attacks discussed have been taken from the scientific literature, and we do not discuss any previously unknown vulnerabilities. All experiments were conducted in a lab, no productive system was jeopardized.

Our stakeholders comprise the *authors of the Northcape capability architecture* [2], *hardware IP developers* working on IP to be integrated with Bredi, *software developers* working on Skadi, members of the *cva6 and Zephyr communities*, and *society at large* in case the technology is widely adopted.

Authors of the Northcape architecture.

Beneficence. Our work provides a proof of feasibility of the proposed concept alongside conceptual improvements. Skadi also extends the security guarantees provided by Northcape.

Respect for Persons. In our paper, we give appropriate credit to Ackermann et al. for their concept and distinguish our improvements to Northcape from the original concept.

Justice. We used the Northcape concept to implement our vision of a combined *trust nothing* hardware-software architecture, leaving different uses of the technology on the table.

Respect for Law and Public Interest. We made sure that our work does not violate any intellectual property rights.

Hardware IP developers.

Beneficence. Bredi isolates SoC components, preventing compromise of memory by untrusted hardware or software.

Respect for Persons. Incompatibilities between Bredi and hardware IP could cause extra verification and development time. Bredi ensures compatibility with *unmodified* hardware IP by maintaining compliance with relevant industry standards, minimizing this harm as much as possible.

Justice. Bredi makes no assumptions about the purpose of devices and attempts to support as many devices as possible.

Respect for Law and Public Interest. Bredi is a purely defensive architecture.

Software developers.

Beneficence. Skadi enforces strong mutual isolation between Zephyr's subsystems, providing increased security.

Respect for Persons. Changes to the Zephyr API could potentially increase development times. We maintained API compatibility as much as possible and minimized changes to the ISA, providing wrapper functions and hardware abstractions to minimize the harms of our changes. We also provide special insecure configurations for easing debugging.

Justice. We ensured compatibility with Zephyr as much as possible, ensuring that existing Zephyr components and applications can be ported without restrictions.

Respect for Law and Public Interest. Skadi is a purely defensive architecture.

cva6 and Zephyr communities.

Beneficence. Our proposed changes to cva6 (for Bredi) and Zephyr (for Skadi) increase the overall security of the system.

Respect for Persons. Improvements to Zephyr and cva6 as well as fixes for any implementation bugs we encountered were offered as pull requests to the respective community. However, Skadi and Bredi will be made available as separate OpenSource project, preventing the risk of *infecting* the code base of the projects with our experimental code. Our separate projects will clearly state their experimental nature, making sure that they are not mistaken for the upstream projects.

Justice. We selected cva6 due to its maturity, sophisticated testbench and implementation in SystemVerilog. We do not claim that implementing Bredi was more or less difficult than in different processors. Likewise, we selected Zephyr as an example of a commercially used RTOS that suited Skadi.

Respect for Law and Public Interest. cva6 and Zephyr are OpenSource, and we followed the licensing agreements.

Society at large.

Beneficence. Skadi and Bredi can contribute to more rigorous security-by-design for embedded devices, improving overall security. Therein, we see a net benefit to society.

Respect for Persons and Justice. Like many confidential computing technologies, Skadi and Bredi could enforce *vendor lock-in*. Our prototype ensures that the device operator retains full control over the device at all times, making sure that our research artifact cannot directly be used for this purpose. We believe that any modification of our results for this purpose are out of scope of this study, as this risk affects every secure computer architecture.

Respect for Law and Public Interest. We believe that increasing the security posture of embedded devices is in the public interest. In this paper, we made both the potential benefits and the limitations of our work transparent. Hence, individuals can make informed decisions whether to deploy a device that utilizes Skadi or Bredi.

Conclusion. We believe that Skadi and Bredi can contribute to increasing security of embedded devices beyond the possibilities of state-of-the-art technologies. At the same time, we have taken appropriate steps to prevent or mitigate the ethical concerns that we identified. Overall, we are confident that the benefits of our work outweigh the potential harms. Hence, we have no concerns about publishing our work.

Open Science

The sources for Bredi and Skadi are available under an OpenSource license [1]. Additionally, we provided scripts and instructions for building and running all experiments included in the paper as well as the console traces and other raw data.

References

[1] Eric Ackermann and Sven Bugiel. Artifact for paper "Trust Nothing: RTOS Security without Run-Time Software TCB" (USENIX Sec'26), 2026. URL https://doi.org/10.5281/zenodo.20259753.

[2] Eric Ackermann, Noah Mauthe, and Sven Bugiel. Work-in-Progress: Northcape: Embedded Real-Time Capability-Based Addressing. In *Workshop on Operating Systems and Virtualization Security*, 2024.

[3] Advanced Micro Devices, Inc. AXI DMA LogiCORE IP Product Guide. LogiCORE IP Product Guide PG021, 2022. URL https://docs.amd.com/r/en-US/pg021_axi_dma.

[4] Advanced Micro Devices, Inc. AXI Interconnect v2.1. LogiCORE IP Product Guide PG059, 2022. URL https://www.xilinx.com/support/documents/ip_documentation/axi_interconnect/v2_1/pg059-axi-interconnect.pdf.

[5] Advanced Micro Devices, Inc. AXI 1G/2.5G Ethernet Subsystem v7.2. LogiCORE IP Product Guide PG138, 2024. URL https://docs.amd.com/r/en-US/pg138-axi-ethernet.

[6] Advanced Micro Devices, Inc. Vivado, 2024. URL https://www.xilinx.com/support/download/index.html/content/xilinx/en/downloadNav/vivado-design-tools/2024-2.html.

[7] Advanced Micro Devices, Inc. AMD Secure Encrypted Virtualization (SEV), 2025. URL https://www.amd.com/en/developer/sev.html.

[8] Hesham Almatary, Michael Dodson, Jessica Clarke, Peter Rugg, Ivan Gomes, Michal Podhradsky, Peter G. Neumann, Simon W. Moore, and Robert N. M. Watson. CompartOS: CHERI Compartmentalization for

Table 11: Detailed Northcape operations.

Syntax	Short Description	Full Description
$create(c_a, l, r, p) \longrightarrow c'_a, c_b$	Split a direct capability into two.	Splits a capability c_a into two non-overlapping pieces. Caller can specify a *smaller* length l as well as *stricter* restrictions r and permissions p for the segment. Requires unlocked and childless direct capabilities.
$merge(c_a, c_b, r, p) \longrightarrow c_m$	Combine two direct capabilities.	Creates a new capability c_m from physically adjacent capabilities c_a, c_b with arbitrary new restrictions r and new permissions p and destroys inputs. Requires direct capabilities that are unlocked and childless.
$derive(c_a, l, o, r, p) \longrightarrow c_i$	Create an indirect capability.	Create indirect capabilities from direct or indirect input capability c_a. The output capability c_i is always indirect and can have an additional offset o and a reduced length l in comparison to its parent.
$clone(c_a, r, p) \longrightarrow c_i$	*derive* that preserves bounds.	Variant of *derive* with an offset o of 0 and a length l that equals the parent. Useful for removing permissions from or adding restrictions to a capability a subsystem wants to share when length l is not known.
$lock(c_a, r, p) \longrightarrow c_l$	Gain exclusive access.	Attempts to gain *exclusive access* to a direct or indirect capability. Creates a lock-holder c_l with exclusive access. Requires *lockable* permission in the grandparent to prevent DoS.
$drop(c_a) \longrightarrow b$	Destroy indirect capability.	Destroys capability c_a. Can only be used on indirect and lock-holder capabilities without references. If c_a is a lock-holder capability returned by *lock*, the parent direct capability is unlocked as a side effect.
$revoke(c_a, r, p) \longrightarrow c'_a$	Destroy direct capability, overwrite segment.	Destroys c_a and creates a new direct capability c'_a for the segment identified by c_a with full permissions and removed restrictions. *Overwrites* the segment with 0-bytes. Returns the new capability. Can only be used on direct capabilities. Permissions p and new restrictions r can be more permissive than those of c_a.
$inspect(c_a) \longrightarrow b, l, r, p$	Determine capability metadata.	Reads the metadata for any capability c_a. Resolves c_a recursively to determine lockable. If c_a has no restrictions or restrictions matching the current subsystem, returns all metadata. If c_a has a set-subsystem-ID restriction, returns *only the* R, W, X, I *permissions* and the restrictions. Otherwise, it returns an error.
$restrict(c_a, r, p, o^+, l^-) \longrightarrow b$	Make capabilities more restrictive in-place.	Makes a capability more restrictive. For indirect capabilities, an offset o^+ and a length minuend l^- can be specified. Permissions that are not specified in *both* p and c_a's metadata are dropped. The lockable permission is only dropped if c_a is a direct capability. r is only applied when c_a does not have restrictions.
$calls(c_a)$	Do a subsystem call.	On input a capability c_a with set-subsystem-id restriction, set the *non-architectural* subsystem identifier register to the indicated identity and transfer control to the code in the identified segment *atomically*.

Embedded Systems, 2022. URL http://arxiv.org/abs/2206.02852.

[9] Saar Amar, David Chisnall, Tony Chen, Nathaniel Wesley Filardo, Ben Laurie, Kunyan Liu, Robert Norton, Simon W. Moore, Yucong Tao, Robert N. M. Watson, and Hongyan Xia. CHERIoT: Complete Memory Safety for Embedded Devices. In *MICRO'23*, 2023.

[10] Saar Amar, Tony Chen, David Chisnall, Nathaniel Wesley Filardo, Ben Laurie, Hugo Lefeuvre, Kunyan Liu, Simon W. Moore, Robert Norton-Wright, Margo Seltzer, Yucong Tao, Robert N. M. Watson, and Hongyan Xia. CHERIoT RTOS: An OS for Fine-Grained Memory-Safe Compartments on Low-Cost Embedded Devices. In *PLOS'25*, 2025.

[11] Arm Ltd. AMBA AXI and ACE Protocol Specification, 2011. URL https://developer.arm.com/documentation/ihi0022/d/.

[12] Arm Ltd. Arm CCA Security Model, 2021. URL https://documentation-service.arm.com/static/610aaec33d73a34b640e333b.

[13] Rafael V. Aroca and Glauco Caurin. A real time operating systems (RTOS) comparison. In *WSO-Workshop de Sistemas Operacionais*, 2009.

[14] Nils Asmussen, Till Miemietz, Sebastian Haas, and Michael Roitzsch. Distrusting cores by separating computation from isolation. *Journal of Systems Architecture*, 159, 2025.

[15] Gal Beniamini. Project Zero: Over The Air: Exploiting Broadcom's Wi-Fi Stack (Part 1), 2017. URL https://googleprojectzero.blogspot.com/2017/04/over-air-exploiting-broadcoms-wi-fi_4.html.

[16] Luca Benini, Alberto Macii, and Massimo Poncino. Energy-aware design of embedded memories: A survey of technologies, architectures, and optimization techniques. *ACM Transactions on Embedded Computing Systems*, 2(1), 2003.

[17] Andrin Bertschi, Supraja Sridhara, Friederike Groschupp, Mark Kuhne, Benedict Schlüter, Clément Thorens, Nicolas Dutly, Srdjan Capkun, and Shweta Shinde. Devlore: Extending Arm CCA to Integrated Devices A Journey Beyond Memory to Interrupt Isolation, 2024. URL http://arxiv.org/abs/2408.05835.

[18] Carl van Schaik and Gernot Heiser. High-performance microkernels and virtualisation on ARM and segmented architectures. In *International Workshop on Microkernels for Embedded Systems*, 2007.

[19] Nicholas P. Carter, Stephen W. Keckler, and William J. Dally. Hardware support for fast capability-based addressing. *Operating Systems Review*, 28(5), 1994.

[20] Haogang Chen, Yandong Mao, Xi Wang, Dong Zhou, Nickolai Zeldovich, and M. Frans Kaashoek. Linux kernel vulnerabilities: State-of-the-art defenses and open problems. In *Asia-Pacific Workshop on Systems*, 2011.

[21] Yueqi Chen and Xinyu Xing. SLAKE: Facilitating Slab Manipulation for Exploiting Vulnerabilities in the Linux Kernel. In *CCS'19*, 2019.

[22] Jianyi Cheng, A. Theodore Markettos, Alexandre Joannou, Paul Metzger, Matthew Naylor, Peter Rugg, and Timothy M. Jones. Adaptive CHERI Compartmentalization for Heterogeneous Accelerators. In *ISCA'25*, 2025.

[23] Kito Cheng and Jessica Clarke. RISC-V ABIs Specification, 2024. URL https://github.com/riscv-non-isa/riscv-elf-psabi-doc/releases/download/draft-20240829-13bfa9f54634cb60d86b9b333e109f077805b4b3/riscv-abi.pdf.

[24] Pau-Chen Cheng, Wojciech Ozga, Enriquillo Valdez, Salman Ahmed, Zhongshu Gu, Hani Jamjoom, Hubertus Franke, and James Bottomley. Intel TDX Demystified: A Top-Down Approach. *ACM Comput. Surv.*, 56(9), 2024.

[25] Yann Collet. Lz4/lz4: Extremely Fast Compression algorithm, 2025. URL https://github.com/lz4/lz4.

[26] Ang Cui, Michael Costello, and Salvatore J. Stolfo. When Firmware Modifications Attack: A Case Study of Embedded Exploitation. In *NDSS'13*, 2013.

[27] Jack B. Dennis and Earl C. Van Horn. Programming semantics for multiprogrammed computations. *Communications of the ACM*, 9(3), 1966.

[28] Kha Dinh Duy, Kyuwon Cho, Taehyun Noh, and Hojoon Lee. Capacity: Cryptographically-Enforced In-Process Capabilities for Modern ARM Architectures. In *CCS'23*, 2023.

[29] Friedrich Doku, Jonathan Laughton, Nick Wanninger, and Peter Dinda. CAPIO: Safe Kernel-Bypass of Commodity Devices using Capabilities, 2025. URL http://arxiv.org/abs/2512.16957.

[30] EEMBC. Coremark. Embedded Microprocessor Benchmark Consortium, 2025. URL https://github.com/eembc/coremark.

[31] Efabless Corporation. Openlane2, 2025. URL https://github.com/efabless/openlane2.

[32] Kevin Elphinstone and Gernot Heiser. From L3 to seL4 what have we learnt in 20 years of L4 microkernels? In *SOSP'13*, 2013.

[33] Lawrence Esswood. *CheriOS: Designing an Untrusted Single-Address-Space Capability Operating System Utilising Capability Hardware and a Minimal Hypervisor*. PhD thesis, University of Cambridge, 2021.

[34] A. Garcia-Martinez, J.F. Conde, and A. Vina. A comprehensive approach in performance evaluation for modern real-time operating systems. In *Euromicro*, 1996.

[35] Google Inc. Skywater PDK, 2025. URL https://github.com/google/skywater-pdk.

[36] Merve Gülmez, Håkan Englund, Jan Tobias Mühlberg, and Thomas Nyman. Mon CHÉRI: Mitigating Uninitialized Memory Access with Conditional Capabilities. In *SP'25*, 2025.

[37] Matthew R. Guthaus, James E. Stine, Samira Ataei, Brian Chen, Bin Wu, and Mehedi Sarwar. OpenRAM: An open-source memory compiler. In *International Conference on Computer-Aided Design*, 2016.

[38] Sebastian Haas, Mattis Hasler, Friedrich Pauls, Christopher Dunkel, Yogesh Verma, Nilanjana Das, and Michael Raitza. An Architectural Approach for the Secure Integration of Hardware Accelerators into a Trustworthy MPSoC Platform. In *International Symposium on Embedded Multicore/Many-core Systems-on-Chip*, 2025.

[39] Matthias Hille, Nils Asmussen, Pramod Bhatotia, and Hermann Härtig. SemperOS: A Distributed Capability System. In *USENIX ATC'19*, 2019.

[40] Haoyang Huang and Fengwei Zhang. HiveTEE: Scalable and Fine-grained Isolated Domains with RME and MTE Co-assisted. *IEEE Transactions on Information Forensics and Security*, 21, 2026.

[41] Henrik Karlsson and Roberto Guanciale. Partitioning Kernel With Capability Controlled Temporal and Spatial Partitioning. In *RTSS'25*, 2025.

[42] Yonghae Kim, Anurag Kar, Jaewon Lee, Jaekyu Lee, and Hyesoon Kim. RV-CURE: A RISC-V Capability Architecture for Full Memory Safety. *IEEE Transactions on Computers*, 74(10), 2025.

[43] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. seL4: Formal verification of an OS kernel. In *SOSP'09*, 2009.

[44] Hugo Lefeuvre, Vlad-Andrei Bădoiu, Ştefan Teodorescu, Pierre Olivier, Tiberiu Mosnoi, Răzvan Deaconescu, Felipe Huici, and Costin Raiciu. FlexOS: Making OS isolation flexible. In *HotOS'21*, 2021.

[45] Hugo Lefeuvre, Nathan Dautenhahn, David Chisnall, and Pierre Olivier. Sok: Software compartmentalization. In *SP'25*, 2025.

[46] Michael LeMay, Joydeep Rakshit, Sergej Deutsch, David M. Durham, Santosh Ghosh, Anant Nori, Jayesh Gaur, Andrew Weiler, Salmin Sultana, Karanvir Grewal, and Sreenivas Subramoney. Cryptographic Capability Computing. In *MICRO'21*, 2021.

[47] Henry M. Levy. *Capability-Based Computer Systems*. 2014.

[48] Zhenpeng Lin, Yuhang Wu, and Xinyu Xing. DirtyCred: Escalating Privilege in Linux Kernel. In *CCS'22*, 2022.

[49] Linux Contributors. Xilinx_axienet_main.c, 2025. URL https://github.com/torvalds/linux/blob/master/drivers/net/ethernet/xilinx/xilinx_axienet_main.c.

[50] A. Theodore Markettos, Colin Rothwell, Brett F. Gutstein, Allison Pearce, Peter G. Neumann, Simon W. Moore, and Robert N. M. Watson. Thunderclap: Exploring vulnerabilities in operating system IOMMU protection via DMA from untrustworthy peripherals. In *NDSS'19*, 2019.

[51] A. Theodore Markettos, John Baldwin, Ruslan Bukin, Peter G. Neumann, Simon W. Moore, and Robert N. M. Watson. Position Paper: Defending Direct Memory Access with CHERI Capabilities. In *HASP'21*, 2021.

[52] John D. McCalpin. Memory bandwidth and machine balance in current high performance computers. *IEEE computer society technical committee on computer architecture (TCCA) newsletter*, 2(19-25), 1995.

[53] Faqiang Mei, Lei Zhang, Chongyan Gu, Yuan Cao, Chenghua Wang, and Weiqiang Liu. A Highly Flexible Lightweight and High Speed True Random Number Generator on FPGA. In *IEEE Computer Society Annual Symposium on VLSI*, 2018.

[54] Paul Metzger, A. Theodore Markettos, Edward Tomasz Napierała, Matthew Naylor, Robert N. M. Watson, and Timothy M. Jones. Deprivileging Low-Level GPU Drivers Efficiently with User-Space Processes and CHERI Compartments. In *CCS'25*, 2025.

[55] Mark S. Miller, Ka-Ping Yee, and Jonathan Shapiro. Capability myths demolished. Technical Report SRL2003-02, Johns Hopkins University Systems Research Laboratory, 2003. URL https://www-users.cse.umn.edu/classes/Fall-2019/csci5271/papers/SRL2003-02.pdf.

[56] Job Noorman, Jo Van Bulck, Jan Tobias Mühlberg, Frank Piessens, Pieter Maene, Bart Preneel, Ingrid Verbauwhede, Johannes Götzfried, Tilo Müller, and Felix Freiling. Sancus 2.0: A Low-Cost Security Architecture for IoT Devices. *ACM Trans. Priv. Secur.*, 20(3), 2017.

[57] Dorottya Papp, Zhendong Ma, and Levente Buttyan. Embedded systems security: Threats, vulnerabilities, and attack taxonomy. In *Annual Conference on Privacy, Security and Trust*, 2015.

[58] David A. Patterson and Andrew Waterman. *The RISC-V Reader: An Open Architecture Atlas*. 2017.

[59] Friedrich Pauls, Sebastian Haas, and Yogesh Verma. A Trust-minimized Platform for Secure AI Accelerator Integration. In *International SoC Design Conference*, 2025.

[60] Francesco Restuccia, Zhenghua Ma, Joseph Zuckerman, Andres Meza, Biruk Seyoum, Luca Carloni, and Ryan Kastner. BASTION: A Framework for Secure Third-Party IP Integration in NoC-based SoC Platforms. *Transactions on Cryptographic Hardware and Embedded Systems*, 2025(4), 2025.

[61] RISC-V International. RISC-V Platform-Level Interrupt Controller Specification, 2023. URL https://docs.riscv.org/reference/plic/_attachments/riscv-plic.pdf.

[62] RISC-V International. Core-Local Interrupt Controller (CLIC) RISC-V Privileged Architecture Extensions, 2025. URL https://github.com/riscv/riscv-fast-interrupt/releases/download/v0.10/clic.pdf.

[63] RISC-V International. The RISC-V Instruction Set Manual: Volume II, 2025. URL https://docs.riscv.org/reference/isa/_attachments/riscv-privileged.pdf.

[64] Vasily A. Sartakov, Stefan Brenner, Sonia Ben Mokhtar, Sara Bouchenak, Gaël Thomas, and Rüdiger Kapitza. EActors: Fast and flexible trusted computing using SGX. In *Middleware'18*, 2018.

[65] Benedict Schlüter, Christoph Wech, and Shweta Shinde. Heracles: Chosen Plaintext Attack on AMD SEV-SNP. In *CCS'25*, 2025.

[66] Alireza Shameli-Sendi. Understanding Linux kernel vulnerabilities. *Journal of Computer Virology and Hacking Techniques*, 17(4), 2021.

[67] Miguel Silva, David Cerdeira, Sandro Pinto, and Tiago Gomes. Operating systems for Internet of Things low-end devices: Analysis and benchmarking. *IEEE Internet of Things Journal*, 6(6), 2019.

[68] Lau Skorstengaard, Dominique Devriese, and Lars Birkedal. Reasoning About a Capability Machine with Local Capabilities. In *Programming Languages and Systems*, 2018.

[69] The MITRE Corporation. MITRE EMB3D, 2025. URL https://emb3d.mitre.org/.

[70] Thomas Van Strydonck, Job Noorman, Jennifer Jackson, Leonardo Alves Dias, Robin Vanderstraeten, David Oswald, Frank Piessens, and Dominique Devriese. CHERI-TrEE: Flexible enclaves on capability machines. In *EuroS&P'23*, 2023.

[71] Lluïs Vilanova, Muli Ben-Yehuda, Nacho Navarro, Yoav Etsion, and Mateo Valero. CODOMs: Protecting software with code-centric memory domains. *ACM SIGARCH Computer Architecture News*, 42(3), 2014.

[72] Lluís Vilanova, Marc Jordà, Nacho Navarro, Yoav Etsion, and Mateo Valero. Direct Inter-Process Communication (dIPC): Repurposing the CODOMs Architecture to Accelerate IPC. In *EuroSys'17*, 2017.

[73] Chenxu Wang, Fengwei Zhang, Yunjie Deng, Kevin Leach, Jiannong Cao, Zhenyu Ning, Shoumeng Yan, and Zhengyu He. CAGE: Complementing Arm CCA with GPU Extensions. In *NDSS'24*, 2024.

[74] Robert N. M. Watson, Jonathan Anderson, Ben Laurie, and Kris Kennaway. Capsicum: Practical capabilities for {UNIX}. In *USENIX Sec'10*, 2010.

[75] Stanley Winkler. *Computer Communication - Impacts and Implications*. Washington, D.C., 1972.

[76] Jonathan Woodruff, Robert N. M. Watson, David Chisnall, Simon W. Moore, Jonathan Anderson, Brooks Davis, Ben Laurie, Peter G. Neumann, Robert Norton, and Michael Roe. The CHERI capability model: Revisiting RISC in an age of risk. In *ISCA'14*, 2014.

[77] Hongyan Xia, Jonathan Woodruff, Sam Ainsworth, Nathaniel W. Filardo, Michael Roe, Alexander Richardson, Peter Rugg, Peter G. Neumann, Simon W. Moore, Robert N. M. Watson, and Timothy M. Jones. CHERIvoke: Characterising Pointer Revocation using CHERI Capabilities for Temporal Memory Safety. In *MICRO'19*, 2019.

[78] Wen Xu, Juanru Li, Junliang Shu, Wenbo Yang, Tianyi Xie, Yuanyuan Zhang, and Dawu Gu. From Collision To Exploitation: Unleashing Use-After-Free Vulnerabilities in Linux Kernel. In *CCS'15*, 2015.

[79] YosysHQ GmbH. Yosys Open Synthesis Suite. Yosys Headquarters, 2025. URL https://github.com/YosysHQ/yosys.

[80] Jason Zhijingcheng Yu, Conrad Watt, Aditya Badole, Trevor E. Carlson, and Prateek Saxena. Capstone: A Capability-based Foundation for Trustless Secure Memory Access. In *USENIX Sec'23*, 2023.

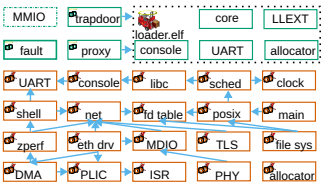


Figure 6: Compartmentalized subsystems in Skadi as well as loader architecture (dashed box) and subsystem calls (arrows).

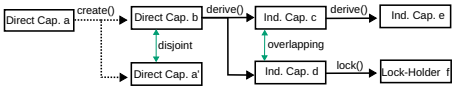


Figure 7: Capabilities created by Northcape operations and their relationships. Solid lines denote tracked parent-child relationships, dashed lines denote implicit relationships.

[81] A. S. Yurchenko. Algorithm of dynamic segmented memory allocation. *Cybernetics*, 17(1), 1981.

[82] Florian Zaruba and Luca Benini. The Cost of Application-Class Processing: Energy and Performance Analysis of a Linux-Ready 1.7-GHz 64-Bit RISC-V Core in 22-nm FDSOI Technology. *Transactions on Very Large Scale Integration Systems*, 27(11), 2019.

[83] Zephyr Project members and individual contributors. Zephyr. Zephyr Project, 2025. URL https://github.com/zephyrproject-rtos/zephyr.

[84] Zephyr Project members and individual contributors. Device Driver Model, 2025. URL https://docs.zephyrproject.org/latest/kernel/drivers/index.html.

[85] Yiming Zhang, Yuxin Hu, Zhenyu Ning, Fengwei Zhang, Xiapu Luo, Haoyang Huang, Shoumeng Yan, and Zhengyu He. {SHELTER}: Extending Arm {CCA} with Isolation in User Space. In *USENIX Sec'23*, 2023.

A Appendix

This appendix provides supplementary material to our system design, implementation and verification.

Table 11 provides a formalized syntax for Northcape operations as well as both a short and a detailed description of operation semantics. Figure 7 illustrates the capability hierarchy in the Northcape system. Figure 6 lists a selection of Skadi subsystems and their interactions.

In our artifact [1], we provide extended tables for our experiments as well as raw data and sources for Skadi and Bredi.